

CS2023 Algoritmos y Estructuras de Datos

Implementación de árbol binario de búsqueda

Semana 6

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

15 de septiembre de 2026

Objetivos de la sesión



Objetivos de la sesión



- ▶ La estructura del nodo

Objetivos de la sesión



- ▶ La estructura del nodo
- ▶ BUSCAR, MÍNIMO, MÁXIMO

Objetivos de la sesión



- ▶ La estructura del nodo
- ▶ BUSCAR, MÍNIMO, MÁXIMO
- ▶ INSERTAR

Objetivos de la sesión



- ▶ La estructura del nodo
- ▶ BUSCAR, MÍNIMO, MÁXIMO
- ▶ INSERTAR
- ▶ SUCESOR y PREDECESOR

Objetivos de la sesión



- ▶ La estructura del nodo
- ▶ BUSCAR, MÍNIMO, MÁXIMO
- ▶ INSERTAR
- ▶ SUCESOR y PREDECESOR
- ▶ ELIMINAR — el más delicado de todos

Objetivos de la sesión



- ▶ La estructura del nodo
- ▶ BUSCAR, MÍNIMO, MÁXIMO
- ▶ INSERTAR
- ▶ SUCESOR y PREDECESOR
- ▶ ELIMINAR — el más delicado de todos
- ▶ RECORRIDO-INORDER

Motivación



La clase pasada quedó en teoría

Vimos el TDA completo y su complejidad $O(h)$ — pero nunca vimos **cómo** se implementa cada operación realmente.

Motivación



La clase pasada quedó en teoría

Vimos el TDA completo y su complejidad $O(h)$ — pero nunca vimos **cómo** se implementa cada operación realmente.

¡Piénsalo!

De las 7 operaciones del TDA, ¿cuál crees que va a ser la más simple de implementar? ¿Y la más delicada?

Motivación



La clase pasada quedó en teoría

Vimos el TDA completo y su complejidad $O(h)$ — pero nunca vimos **cómo** se implementa cada operación realmente.

¡Piénsalo!

De las 7 operaciones del TDA, ¿cuál crees que va a ser la más simple de implementar? ¿Y la más delicada?

Spoiler

BUSCAR es casi trivial. ELIMINAR — por los casos que hay que manejar — es, de lejos, la más delicada. Hoy construimos las 7, en ese orden de dificultad creciente.

La estructura

El nodo



Campos

Cada nodo x guarda:

El nodo



Campos

Cada nodo x guarda:

- ▶ $x.clave$: el valor almacenado.

El nodo



Campos

Cada nodo x guarda:

- ▶ $x.clave$: el valor almacenado.
- ▶ $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).

El nodo



Campos

Cada nodo x guarda:

- ▶ $x.clave$: el valor almacenado.
- ▶ $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).
- ▶ $x.padre$: puntero al padre (nulo si x es la raíz).

El nodo



Campos

Cada nodo x guarda:

- ▶ $x.clave$: el valor almacenado.
- ▶ $x.izq$, $x.der$: punteros a los hijos (nulos si no existen).
- ▶ $x.padre$: puntero al padre (nulo si x es la raíz).

¡Piénsalo!

En la lista doblemente enlazada, guardar el puntero al anterior nos ahorra $O(n)$ al eliminar. ¿Qué nos va a ahorrar $x.padre$ aquí?

Búsqueda y extremos

Buscar



Algoritmo 1: BUSCAR(x, k)

si $x = \text{NULO}$ **o** $k = x.clave$ **entonces**

devolver x

si $k < x.clave$ **entonces**

devolver BUSCAR($x.izq, k$)

devolver BUSCAR($x.der, k$)

Buscar: la idea



Exactamente la traducción directa de la propiedad BST: compara, y sigue por el lado correcto. $O(h)$: baja, a lo más, un camino completo.

Mínimo y Máximo



Algoritmo 2: MÍNIMO(x)

mientras $x.izq \neq \text{NULO}$ **hacer**

$x \leftarrow x.izq$

devolver x

Mínimo y Máximo: por qué funciona



¡Piénsalo!

¿Por qué basta con ir siempre a la izquierda, sin comparar ningún valor?

Mínimo y Máximo: por qué funciona



¡Piénsalo!

¿Por qué basta con ir siempre a la izquierda, sin comparar ningún valor?

Respuesta

Por la propiedad BST, todo lo que está a la izquierda de un nodo es **menor** — el nodo más a la izquierda no tiene nada menor que él. **MÁXIMO** es simétrico, yendo siempre a la derecha.

Insertar

Insertar



Insertar: la idea



¡Piénsalo!

El ciclo `while` se parece mucho a `BUSCAR`. ¿Qué está buscando en realidad, si `k` **no** está todavía en el árbol?

Insertar: la idea

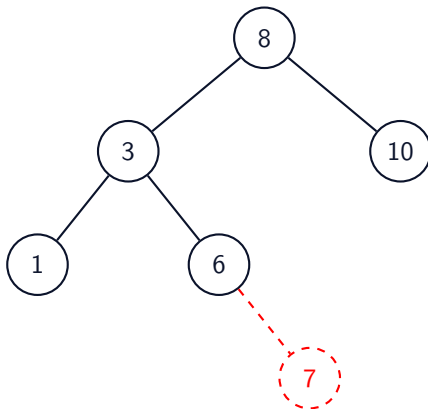
¡Piénsalo!

El ciclo `while` se parece mucho a `BUSCAR`. ¿Qué está buscando en realidad, si `k` **no** está todavía en el árbol?

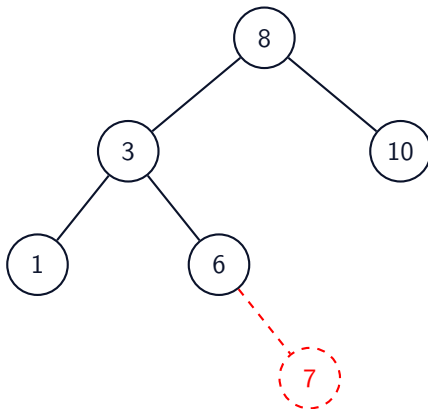
Respuesta

Busca el lugar donde **debería** estar: el ciclo termina justo cuando cae en un puntero nulo — ahí es donde cuelga el nodo nuevo, como hijo de `y` (el último nodo visitado).

Insertando, paso a paso



Insertando, paso a paso



Insertando 7: $8 \rightarrow$ izquierda ($7 < 8$); $3 \rightarrow$ derecha ($7 > 3$); $6 \rightarrow$ derecha ($7 > 6$);
nulo $\rightarrow$ aquí cuelga el nuevo nodo, como hijo derecho de 6.

Sucesor y Predecesor

Sucesor



Algoritmo 3: SUCESOR(x)

```
si  $x.der \neq \text{NULO}$  entonces
    devolver MÍNIMO( $x.der$ )
 $y \leftarrow x.padre$ 
mientras  $y \neq \text{NULO}$  y  $x = y.der$  hacer
     $x \leftarrow y$ 
     $y \leftarrow y.padre$ 
devolver  $y$ 
```

Sucesor: dos casos



¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Sucesor: dos casos



¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Caso 1: hay hijo derecho

El sucesor es el **mínimo** de ese subárbol derecho — el siguiente valor mayor siempre está ahí.

Sucesor: dos casos

¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Caso 1: hay hijo derecho

El sucesor es el **mínimo** de ese subárbol derecho — el siguiente valor mayor siempre está ahí.

Caso 2: no hay hijo derecho

Sube por los punteros *padre* mientras vengas de un hijo **derecho**; el primer ancestro del que vengas por la **izquierda** (o la raíz misma) es el sucesor.

Sucesor: dos casos

¡Piénsalo!

Si x **tiene** hijo derecho, ¿dónde está su sucesor? Y si **no** tiene hijo derecho, ¿cómo lo encuentras sin volver a bajar desde la raíz?

Caso 1: hay hijo derecho

El sucesor es el **mínimo** de ese subárbol derecho — el siguiente valor mayor siempre está ahí.

Caso 2: no hay hijo derecho

Sube por los punteros *padre* mientras vengas de un hijo **derecho**; el primer ancestro del que vengas por la **izquierda** (o la raíz misma) es el sucesor.

PREDECESOR es exactamente simétrico: izquierda en vez de derecha, en todos los pasos.

Eliminar

Eliminar: tres casos



¡Piénsalo!

Antes de ver código: si el nodo z que quieres eliminar tiene 0, 1, o 2 hijos, ¿en qué se complica cada caso?

Eliminar: tres casos

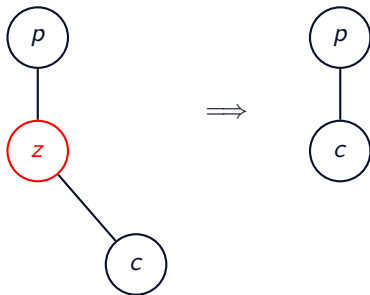


¡Piénsalo!

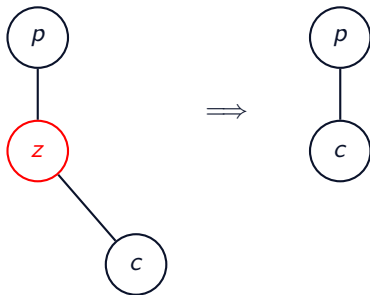
Antes de ver código: si el nodo z que quieres eliminar tiene 0, 1, o 2 hijos, ¿en qué se complica cada caso?

Con 0 o 1 hijo, es fácil: el padre de z simplemente “adopta” a ese único hijo (o a nada). El problema real es con 2 hijos: no puedes simplemente conectar los dos directamente al padre de z .

Caso 1 y 2: cero o un hijo



Caso 1 y 2: cero o un hijo



El padre de z pasa a apuntar directamente a c (el único hijo de z , o nulo si no tenía ninguno) — z simplemente se “salta”.

El helper: Transplantar



Algoritmo 4: TRANSPLANTAR(T, u, v)

```
// Reemplaza el subárbol de  $u$  por el de  $v$ 
si  $u.padre = \text{NULO}$  entonces
    |  $T.raiz \leftarrow v$ 
si no, si  $u = u.padre.izq$  entonces
    |  $u.padre.izq \leftarrow v$ 
en otro caso
    |  $u.padre.der \leftarrow v$ 
si  $v \neq \text{NULO}$  entonces
    |  $v.padre \leftarrow u.padre$ 
```

Transplantar: para qué sirve



No mueve datos: solo reconecta punteros — exactamente lo que necesitábamos en el caso de 0 o 1 hijo.

Caso 3: dos hijos



¡Piénsalo!

No puedes simplemente conectar ambos hijos de z al padre de z (¿a dónde iría el segundo?). ¿Qué **otro** valor del árbol podría ocupar el lugar de z sin romper la propiedad BST?

Caso 3: dos hijos



¡Piénsalo!

No puedes simplemente conectar ambos hijos de z al padre de z (¿a dónde iría el segundo?). ¿Qué **otro** valor del árbol podría ocupar el lugar de z sin romper la propiedad BST?

Respuesta: el sucesor de z

El $\text{SUCEBOR}(z)$ — el mínimo del subárbol derecho — es el valor **más cercano** a z por arriba. Ponerlo en el lugar de z mantiene el orden: nada a la izquierda es mayor, nada a la derecha es menor.

Caso 3: dos hijos



¡Piénsalo!

No puedes simplemente conectar ambos hijos de z al padre de z (¿a dónde iría el segundo?). ¿Qué **otro** valor del árbol podría ocupar el lugar de z sin romper la propiedad BST?

Respuesta: el sucesor de z

El $\text{SUCEADOR}(z)$ — el mínimo del subárbol derecho — es el valor **más cercano** a z por arriba. Ponerlo en el lugar de z mantiene el orden: nada a la izquierda es mayor, nada a la derecha es menor.

Y como es un **mínimo**, el sucesor nunca tiene hijo izquierdo — eliminarlo de su posición original cae en el Caso 1 o 2, que ya resolvimos.

Eliminar



Eliminar: leyendo el caso 3 con cuidado



¡Piénsalo!

¿Por qué el código distingue si $y.padre = z$ (el sucesor es **hijo directo** de z) del caso general?

Eliminar: leyendo el caso 3 con cuidado



¡Piénsalo!

¿Por qué el código distingue si $y.padre = z$ (el sucesor es **hijo directo** de z) del caso general?

Respuesta

Si y ya es el hijo derecho directo de z , no hace falta “sacarlo” de ningún lado — simplemente se convierte en la nueva raíz del subárbol. Si y está **más abajo**, primero hay que desconectarlo de su posición original ($\text{TRANSPLANTAR}(T, y, y.der)$) antes de moverlo.

Recorrido inorder

Recorrido-Inorder



Algoritmo 5: RECORRIDO-INORDER(x)

si $x \neq \text{NULO}$ entonces

 RECORRIDO-INORDER($x.\text{izq}$)

 visitar x

 RECORRIDO-INORDER($x.\text{der}$)

Recorrido-Inorder: por qué funciona



Tres líneas, y por la propiedad BST (que vimos la clase pasada) esto **siempre** visita los elementos en orden creciente. $\Theta(n)$: visita cada nodo exactamente una vez.

Cierre de la sesión



- ▶ El nodo guarda *clave*, *izq*, *der* y *padre* — este último es lo que permite SUCESOR y ELIMINAR sin volver a bajar desde la raíz.
- ▶ BUSCAR, MÍNIMO/MÁXIMO e INSERTAR son variaciones directas de “sigue la propiedad BST hasta un puntero nulo”.
- ▶ SUCESOR tiene dos casos: mínimo del subárbol derecho si existe, o subir hasta el primer ancestro del que vengas por la izquierda.
- ▶ ELIMINAR tiene tres casos (0, 1 o 2 hijos); el caso de 2 hijos se resuelve reemplazando *z* por su SUCESOR — que, al ser un mínimo, siempre cae en el Caso 1 o 2 al eliminarlo de su lugar original.
- ▶ RECORRIDO-INORDER es recursivo y directo: izquierda, visitar, derecha.

Logros de la sesión



Logros de la sesión



- ▶ Sabemos implementar las 7 operaciones del TDA BST con pseudocódigo real.

Logros de la sesión



- ▶ Sabemos implementar las 7 operaciones del TDA BST con pseudocódigo real.
- ▶ Sabemos por qué guardar $x.padre$ simplifica SUCESOR y ELIMINAR.

Logros de la sesión



- ▶ Sabemos implementar las 7 operaciones del TDA BST con pseudocódigo real.
- ▶ Sabemos por qué guardar $x.padre$ simplifica SUCESOR y ELIMINAR.
- ▶ Sabemos manejar los 3 casos de ELIMINAR, incluyendo por qué el caso de 2 hijos se reduce a los otros dos vía el sucesor.

Gracias